

μ T-Kernel で音響処理と NPU 推論を両立する 音響イベント検出基盤の開発

STM32N6570-DK + μ T-Kernel 3.0 (BSP2)

辻 大知

福井工業高等専門学校 専攻科 生産システム工学専攻

1:プロジェクト概要

マイコン 1 個で、音を 1 サンプルも落とさずに NPU 推論を回す基盤を開発



何のために

- NPU 付きマイコンで、音の取り込みと推論を同時に動かす
- 推論が重くても遅れても、音は壊さない
- それを「動いた」ではなく、数字で示す



何を作ったか

- μ T-Kernel のタスク構成（音 > 推論 > 表示）
- NPU・フラッシュ・LCD の初期化と前処理
- 音の判定と通知（LCD・LED・UART）
- 自己検証と、ログの集計・再生の道具



動作例：部屋の音（犬・泣き声・くしゃみ など）を検知して、画面と LED で知らせる（8 章）

この章の流れ

1-1 なぜ難しいか



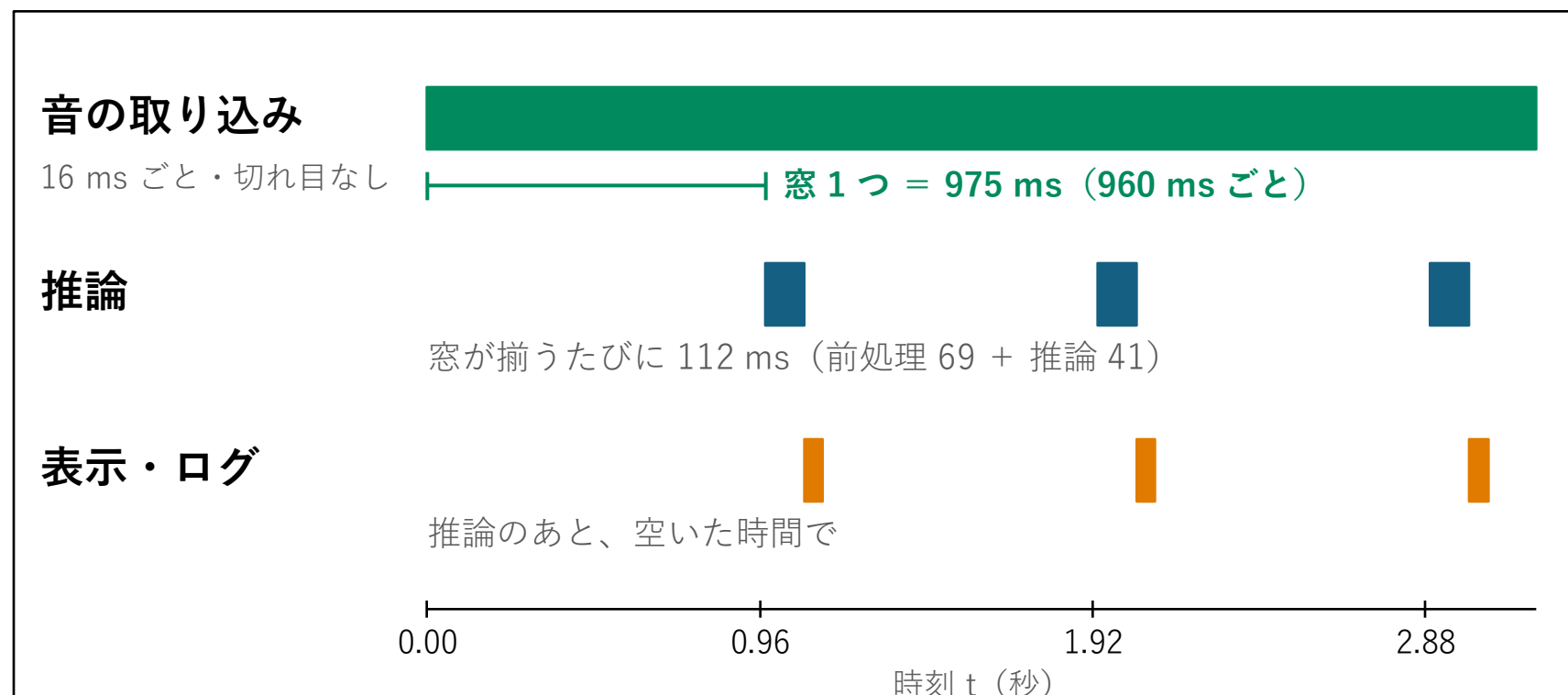
1-2 作ったもの



1-3 使った道具

1-1 なぜ難しいか：1 コアで 3 つの仕事

音は 16 ms ごとに受け取らないと溢れる。
そこへ 1 回 112 ms の推論が割り込む



取りこぼしの数え方

under ヘッドホンに渡す音が足りず、無音で埋めた回数

over マイクの音を受け取れずに捨てた回数

late 処理が 1 周期ぶん間に合わなかった回数

3つの指標とも0ならば
1 サンプルも落ちていない。

以下「音の取りこぼし」はこの 3 つを指す

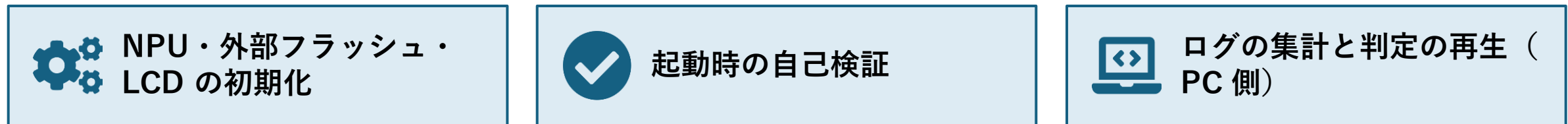
取り込みが落ちると、検出器に入る音そのものが欠ける。
→音を最優先にした (2 章)

1-2 システムフロー：マイクから通知まで



土台： μ T-Kernel 3.0 のタスク・優先度・ハンドラ（既存）

支える部分



既存（ハード・OS・ST の推論エンジンと学習済みモデル）

自作

マイクから通知まで一本道。OS と推論エンジン・学習済みモデル以外は自作した

1-3 使ったツール

市販の評価ボードと無償のツールだけで作成。

揃えれば同じものが作れる



STM32N6570-DK
Cortex-M55 600 MHz ・ NPU 300 MHz
PDM マイク ・ 800 × 480 LCD



μT-Kernel 3.0 (BSP2)

OS：タスク・優先度・ハンドラ



STM32CubeIDE 2.2.0

ビルド、デバッガからの実行、フラッシュ書き込み



ST Edge AI Core 4.0.1

学習済みモデルを NPU 用のコードに変換



ST model zoo (YAMNet)

学習済みの int8 モデル



VS Code ・ Git / GitHub

編集・履歴・公開



Python (uv) ・ PowerShell

ログの取得・集計・判定の再生

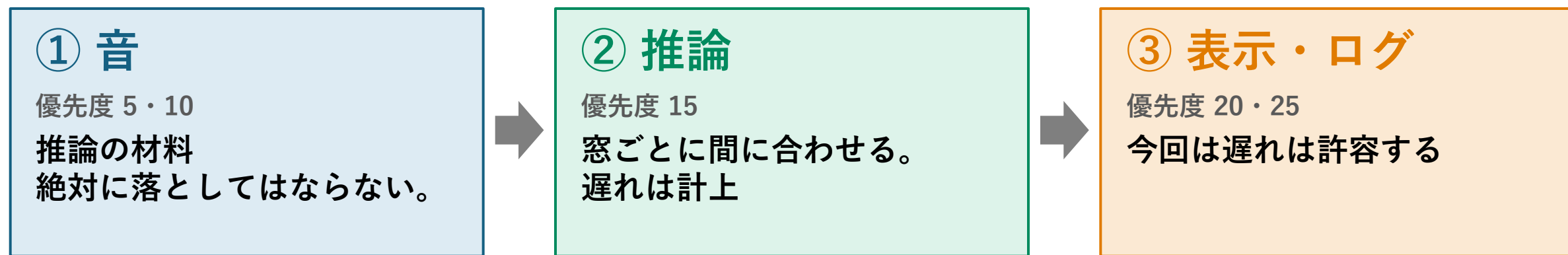


ESC-50

試験に流す音のクリップ

2 設計：音を最優先にする

音 > 推論 > 表示 の順で優先度を固定し、
壊れるなら表示から壊れるようにした



ゆえに4章で...



壊して測った (実験 1)



長時間回した (実験 2)



モデルを替えて同じ手順
で測った (実験 3)

この章の流れ

2-1 タスクと優先度



2-2 μ T-Kernel の機能でつなぐ



2-3 時間と資源の配分

2-1 タスクと優先度

音のタスクが常に最優先

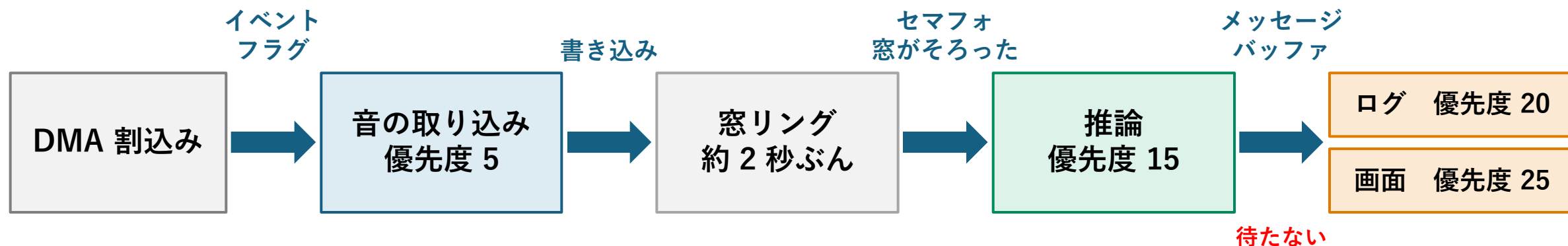
画面が一番下で、空いた時間に動く

高	優先度	仕事（タスク名）	起きるきっかけ	周期・時間
	5	音の取り込み task_pcm	DMA 割込み (イベントフラグ)	16 ms ごと（マイク）
	10	音の見張り task_audio	500 ms ごとのタイマ	取りこぼしの統計を出す
	15	推論 task_infer	窓が揃ったら	960 ms ごとに 112 ms
	20	ログ書き出し reporter	ログが届いたら	UART に書くのはここだけ
	25	画面 task_lcd	通知が届いたら	通知のときだけ
低	その他 生存表示 task_1 (10) : 緑 LED を 500 ms で点滅 / トレース出力 dump (32) 周期ハンドラ : 1 秒ごとに統計を 1 行 / アラームハンドラ : LED を 1 秒後に消す			

2-2 μ T-Kernel の機能でつなぐ

割込みはフラグで起こし、下流へは待たずに渡す。

下の仕事は上の仕事を止められない



周期ハンドラ (1 秒ごと) → 統計を 1 行 / アラームハンドラ → LED を 1 秒後に消す / 時刻は tk_get_otm

固定優先度

推論やログが音を押しのけない。逆にすると音が壊れた (実験 1 ②)

tk_cre_tsk (優先度を指定)

イベントフラグ

割込みは起こすだけ。処理はタスクで行うので、割込みが短い

tk_set_flg / tk_wai_flg

待たないメッセージバッファ

受け側が満杯なら捨てて数える。送り側は待たされない

tk_snd_mbf (TMO_POL) / tk_rcv_mbf

2-3 時間とリソースの配分

推論は周期 960 ms の 12%。残り 88% の時間が音と表示に回る

1 周期 960 ms の使い方

前処理 69 ms



推論 41 ms (計算 9 + 重みの読み出し 32)

音が鳴ってから知らせるまで：1.1～1.9 秒 (実測)



処理 0.112 s

遅れの大半は窓の長さと区切り待ちで決まる (理論 1.07～2.03 s)。RTOS 側の処理は 0.112 s

メモリの置き場所

AXISRAM1・2	1 MB	プログラムと作業領域
AXISRAM3	384 KB	LCD の画面
AXISRAM4	148 KB	FSD50K 版の重み
AXISRAM6	144 KB	NPU の作業域
外部フラッシュ	3.1 MiB	ESC-10 版の重み

プログラム 372 KB (ESC-10 版)

→省電力は未計測。空いている 88% の時間に待機命令 (WFI) を入れるのが次の一手 (9 章)

3 工夫：無いものは作り、作ったものは確かめる

**BSP2 に無かった初期化と前処理を自作し、
起動のたびに正しさを確かめる**



NPU・外部フラッシュ・LCD の
初期化

4 段

すべてレジスタを読み戻して一致を確認

外部フラッシュ → NPU → LCD → フォールト基盤



音を画像にする前処理

差 0 / 12,288

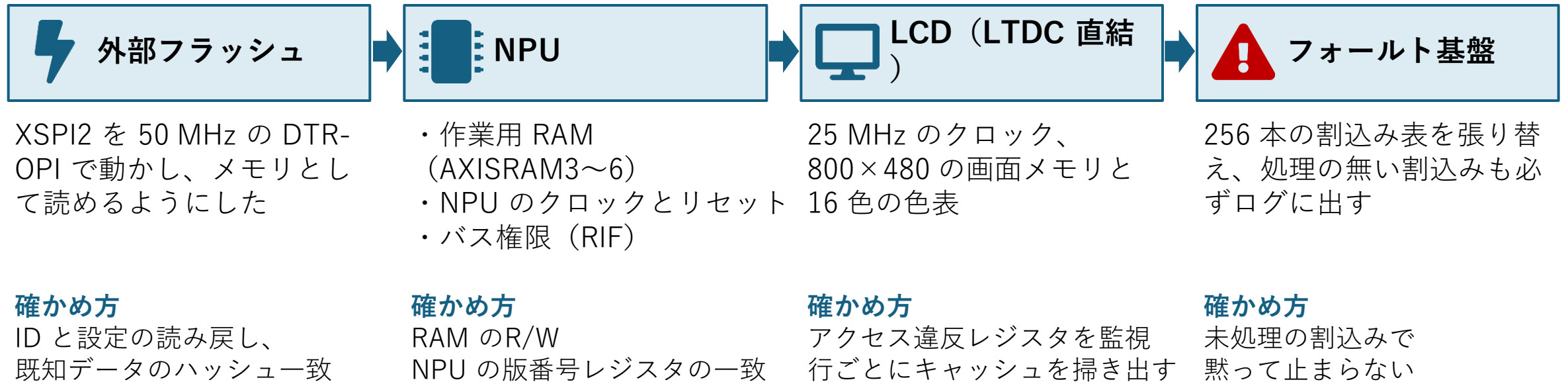
板と PC の出力が 1 ビットも違わない

64 mel × 96 フレーム、int8 まで比較



3-1 BSP2 に無かった 4 段の初期化

初期化は 4 段。権限（RIF）を入れないと、黙って 0 が返る

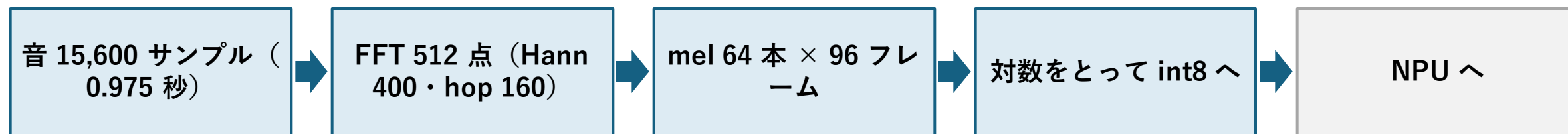


罨：バス権限（RIF）が無いと、読み出しは黙って 0 になり、フォルトも出ない。
アクセス違反レジスタ（RISAF の IASR=0x2）を見つけるまで、LCD は真っ黒だった

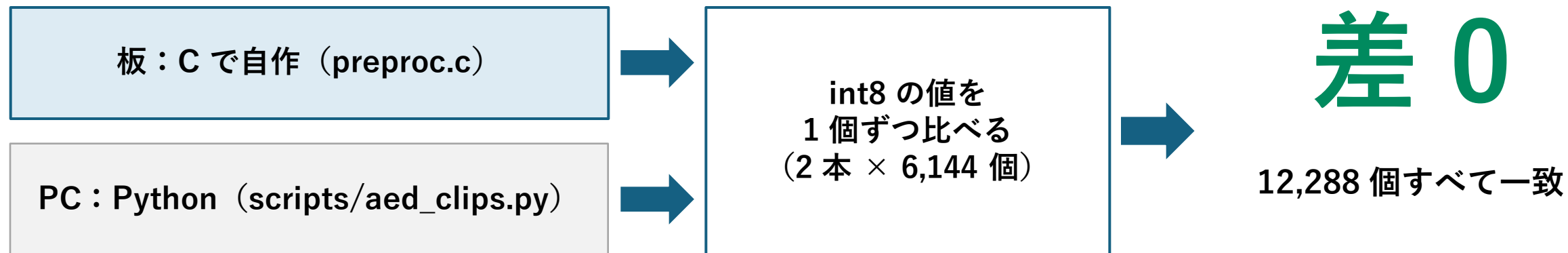
3-2 前処理を自作し、PC と 1 ビットも違わなくした

板の前処理は PC と 12,288 個すべて一致。

入力を疑わなくてよい



同じ音クリップ 2 本 (ESC-50)



起動のたびにこの比較を自動で行い、結果を READY の前に出す (10-2。比べる音は ESC-50 から作る)。
推論にかかる時間は(2-3)

4 検証の方法

設計どおりかは実測でしか言えない。

一段ずつ確かめ、最後は 3 つの実験で測った



実験 1 対照試験

わざと壊す

問い

優先度を逆に、
また推論が周期を超えたら
何が先に壊れるか

→ 5 章



実験 2 長時間運転

1 時間止めない

問い

音・ログ・画面が
欠けないか。
実際の部屋で誤報が
何件出るか

→ 6 章



実験 3 モデル差替え

別のモデルを載せる

問い

同じ手順が通るか。
基盤の数字は変わらないか

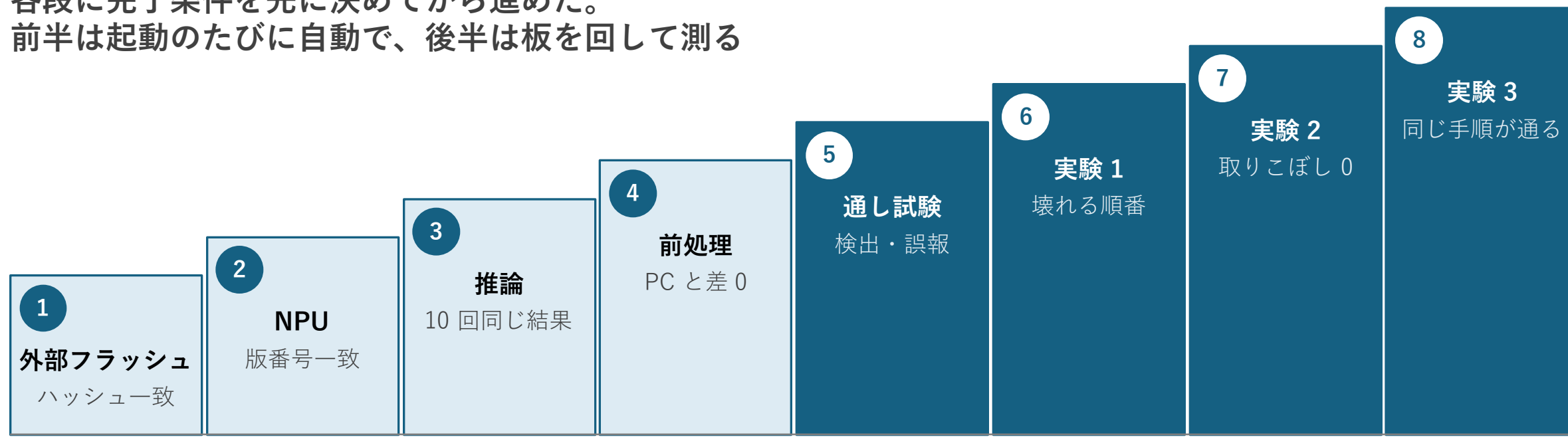
→ 7 章



4-1 一段ずつ確かめる

前の段が通っていれば、次の段の失敗は次の段のせいだと分かる

各段に完了条件を先に決めてから進めた。
前半は起動のたびに自動で、後半は板を回して測る

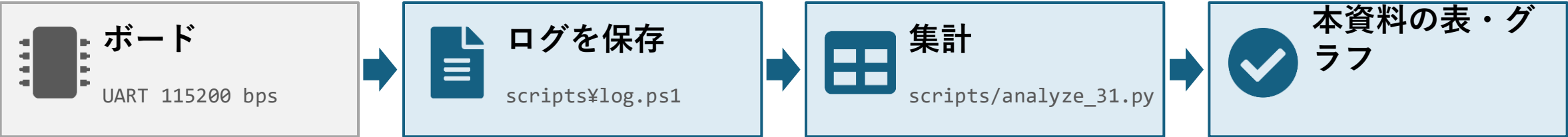


起動のたびに自動（1～4）。結果は READY の前に出る

板を回して測る（5～8）。UART のログを保存して集計

4-2 数字はすべてログから作り直せる

ログは全部残し、表とグラフはスクリプトで作直す



 **判定の再生** scripts/notify_replay.py ログの窓行から板と同じ判定を PC で計算、板の判定と照合（1 窓を除き一致）

本資料の数字	元のログ（docs/logs/）
実験 1（5 章）	①② は A・B のログ、③ は uart_20260927_013950
実験 2：対策前 1,216 件/時（6-2）	uart_20260927_130250
実験 2：1 時間運転（6-1）	uart_20260927_161828
実験 2：対策後・通し試験（6-3）	uart_20260928_002702 + play_20260928_002821
実験 3（7 章）	uart_20260928_064258 + play_20260928_064446
前処理の一致（3-2）	各起動ログの preproc test の行

5 実験 1：対照試験（わざと壊す）

壊れる場所は優先度の並びだけで決まり、設計どおり表示から壊れた

	① 通常	② 推論を音より優先	③ 推論を 10 倍重く
音（取り込み）	 無事	 8% 欠落 途切れて聞こえる	 取りこぼし 0
推論	 無事	 無事（判定も同じ）	 窓の 13% が消えた
表示・ログ	 無事	 無事	 画面 10 分停止 ログ 93% を捨てた

この章の流れ

5-1 3 つの条件



5-2 結果の数字



5-3 ② の詳細



5-4 ③ の詳細

5-1 3つの条件

① 通常に対し、② は優先度を逆に、③ は推論を重くした

① 通常

上ほど優先

音 5・10
推論 15
表示・ログ 20・25

② 推論を音より優先

上ほど優先

推論 3
音 5・10
表示・ログ 20・25

③ 推論を 10 倍重く

上ほど優先

音 5・10
推論 15 (重さ 10 倍) 1 窓 1,103 ms > 周期 960 ms
表示・ログ 20・25

予想：表示から壊れ、音は無事

10 分ずつ、同じ部屋・同じ音源で比べた。切替はビルド時のマクロ 2 つだけで、ほかのコードは同じ。
起動ログの CONFIG 行に条件が出る (ログ上の名前：①=A、②=B、③=D)

5-2 結果の数字

結果として

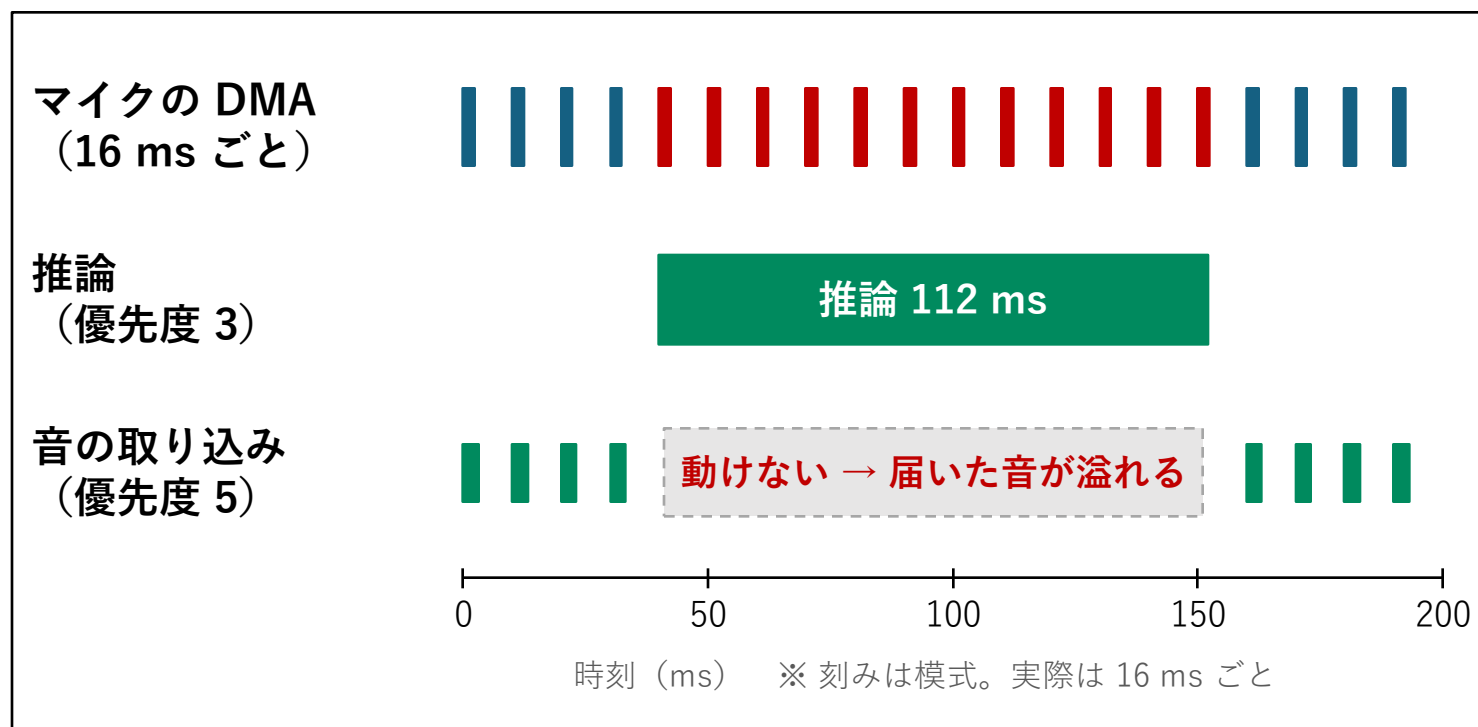
② では音だけが壊れ、③ では音だけが無事だった

	① 通常	② 推論を音より優先	③ 推論を 10 倍重く
音の取りこぼし (under/over/late)	0 / 0 / 0	2,078 / 1,200 / 1,200	0 / 0 / 0
入力の速さ (本来 16,000 Hz)	16,128 Hz	14,848 Hz (8% 欠落)	16,128 Hz
推論できなかった窓	0	0	13%
捨てたログ	0	0	93%
画面	動作 (遅れ最大 102 ms)	動作	10 分間止まった
ヘッドホンの聞こえ方	正常	途切れる	正常

前処理 69 ms・推論 41 ms と判定の結果は 3 条件で同じ。変わったのは「間に合うか」だけ

5-3 ② 推論を音より優先すると、なぜ音が壊れるか

推論の 112 ms の間、音のタスクが動けず、届いた音が溢れる



結果 (10 分)

音の取りこぼし (under / over / late)

2,078 / 1,200 / 1,200

入力の速さ (本来 16,000 Hz)

14,848 Hz (8% 欠落)

窓バッファへの書き込みの最大待ち

414 ms

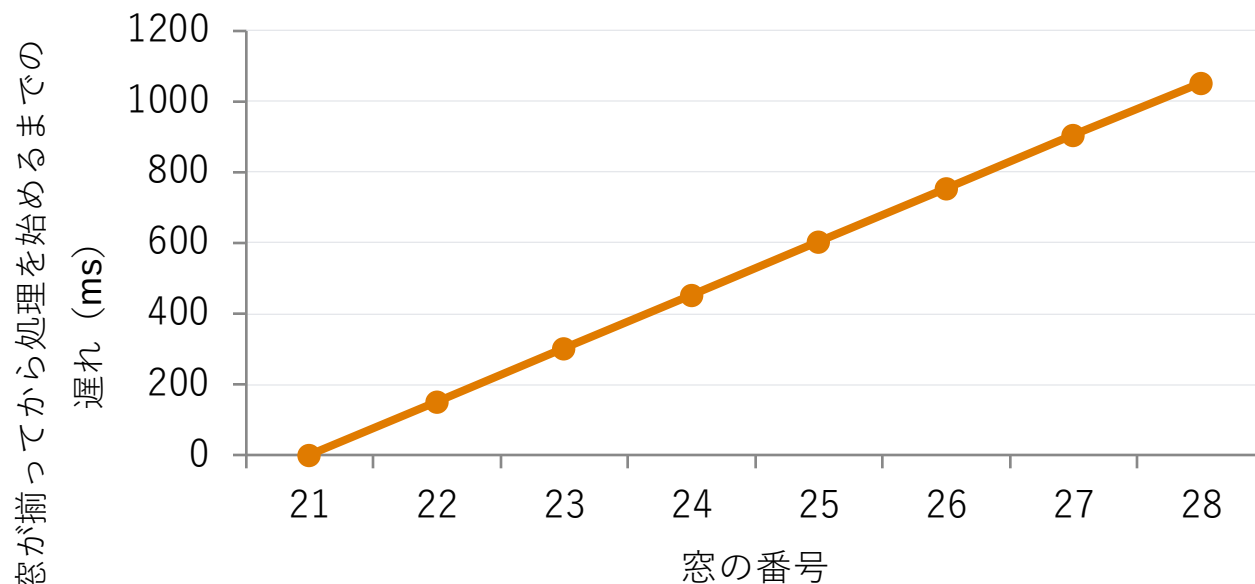
over・late は窓ごとに 2 回ずつ積み上がった
(600 窓 × 2 = 1,200)

推論・画面・ログは無事で、判定も ① と同じ。ヘッドホンでは音が途切れて聞こえる

5-4 ③ 推論を 10 倍重くすると、表示から壊れる

遅れが 150 ms ずつ積み上がり、**画面が停止**
音は流れる。

1,053 ms で窓バッファを追い越し、窓が消える



音の取りこぼし

0 回

消えた窓

13%

捨てたログ

93%

画面

10 分停止

1 窓に 1,103 ms かかり、周期 960 ms を超える。優先度の低いログと画面は 598 秒間一度も動けず、推論が止まった瞬間に溜まった分が一気に流れた。音のタスクは上にいるので、周期どおり動き続けた

6 実験 2：長時間運転（1 時間）

1 時間、音もログも画面も一つも落とさなかった。
誤報は 1,216 → 3.5 件/時に減らした

音の取りこぼし

0 回

推論できなかった窓

0 / 3,854

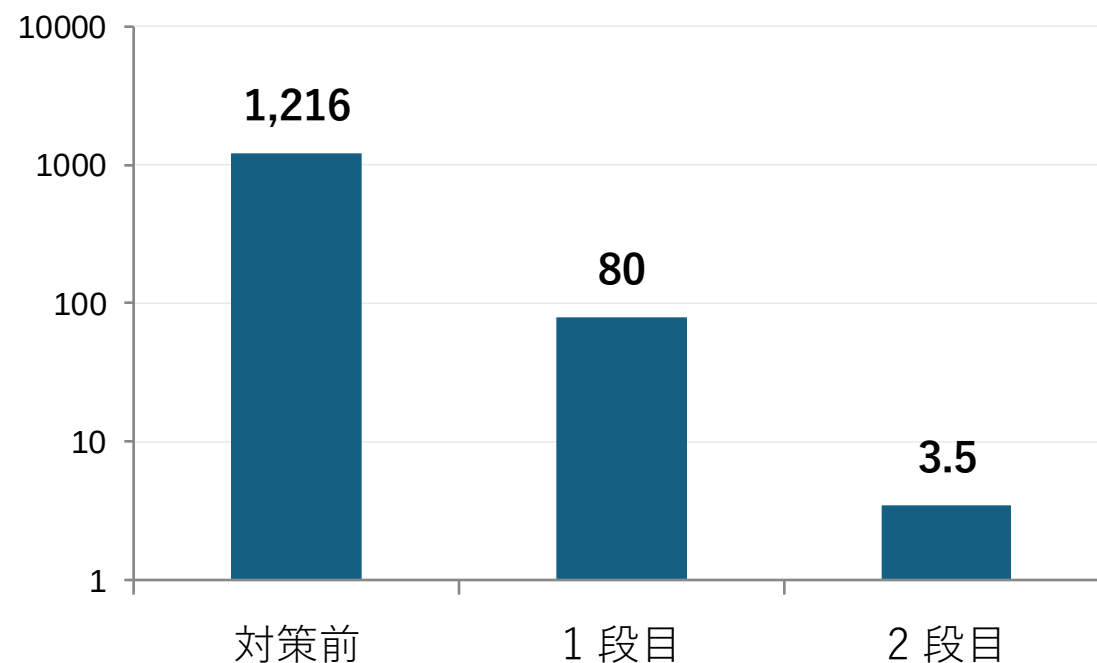
ログ 24,238 行

欠け 0

画面の更新 83 回

欠け 0

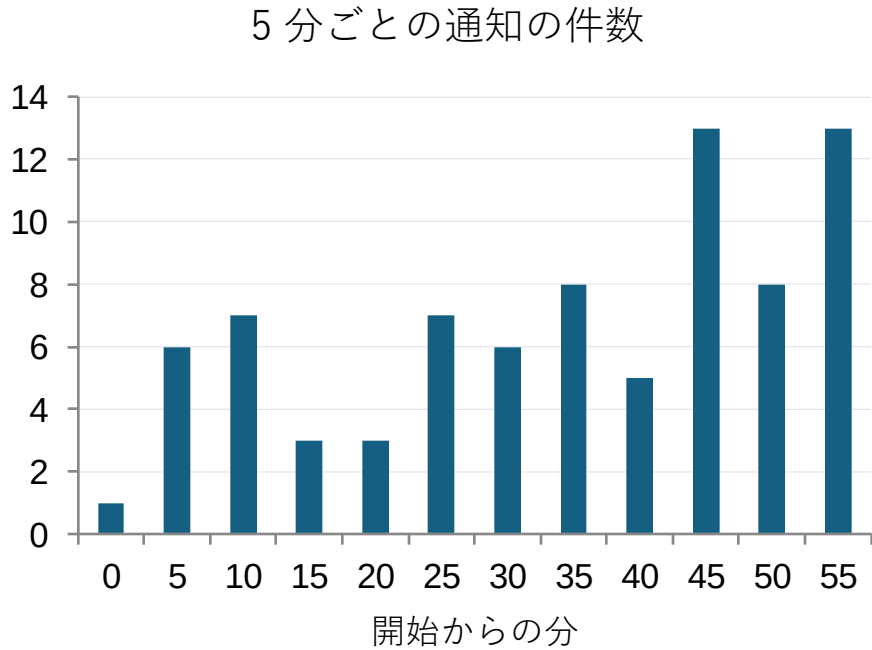
誤報（件/時、縦軸は対数）



6-1 1 時間の結果

基盤は 1 時間まったく劣化しない。
一方で誤報が 80 件/時出た

項目	値
推論した窓	3,854 / 3,854（できなかった窓 0）
音の取りこぼし	0 / 0 / 0（書き込み 231,287 回、最大 10.5 μs）
前処理 / 推論	平均 69.08 / 41.05 ms（最大との差 0.1 ms 以内）
検出から通知まで	最大 112.3 ms
ログ	24,238 行、欠け 0（遅れ最大 128 ms）
画面	83 回更新、欠け 0（遅れ最大 98.7 ms）



2026-09-27 16:18～17:20、冷蔵庫のある生活空間・在室、ESC-10 版、対策 1 段目の規則。
通知 80 件はすべて何も鳴っていない窓（平均音量 -44～-33 dBFS）だった

6-2 誤報の原因と 2 段の対策

固定の音量ではなく、その部屋の暗騒音との差で切る



原因

冷蔵庫の低い連続音（平均 -48 dBFS）をモデルが「くしゃみ」と判定した。無音の実験室では 0 件だったのに、部屋で 10 分回すと 199 件（1,216 件/時）

対策前の 10 分のログに規則を当て、残る件数を PC で数えた（判定の再生）

	確率 0.5 以上	0.7 以上	0.8 以上
音量の条件なし	202	75	25
瞬間音量 - 30 dBFS 以上	17	6	1
+ 暗騒音より 10 dB 以上大きい	8	3	0

太枠が採用した組み合わせ（上：1 段目、下：2 段目）。
起動から 1 分は相対の条件を使わない

1 段目：確率 0.7 以上 + 瞬間音量 - 30 dBFS 以上
→ 1 時間で 80 件/時（6-1）



2 段目：+ 直近 1 分の暗騒音（音量の下位 10%）より 10 dB
大きい音だけ通す → 3.5 件/時（6-3）

6-3 対策の効き目

2,117 窓のうち通知まで届いたのは、実際に鳴った 2 回だけ

2,117

34 分のすべての窓

53

モデルが対象の音だと確信した窓（確率 0.7 以上）

19

そのうち瞬間音量 - 30 dBFS 以上（1 段目で残る）

2

そのうち暗騒音より 10 dB 以上大きい = 通知

2 段目が無ければ 19 回（34 件/時）通知していた。
残った 2 回は暗騒音 +19 dB・+13 dB の実際の物音

通し試験（同じ夜・同じ規則）



無音 60 秒

通知 0



対象の音のクリップ 6 本

4 本を検出（2 本は音量過大でマイクが飽和）



対象外のクリップ 4 本

通知 0



対象外の生活音 9 回

誤報 0（対策前は 12 回中 1）



板の判定と PC の再生

1 窓を除き一致（起動直後）

7 実験 3：モデル差し替え

1 行の切替で同じ検証が通り、基盤の数字は変わらなかった。弱点はモデル側に出た

判定・通知・ログ

モデル：ESC-10 版 ⇄ FSD50K 版
(AED_MODEL を 1 行)

前処理 (log-mel)

タスク構成・ μ T-Kernel

ボード STM32N6570-DK

オレンジだけを替えた

基盤の数字：変わらない

音の取りこぼし 0 → 0

自己検証・前処理の一致 通過 → 通過

推論の時間 41.1 → 3.55 ms

モデルの数字：悪化

生活音の誤報 0/9 → 9/10

放置中の通知 3.5 → 250 件/時

この章の流れ

7-1 何を変えたか



7-2 結果

7-1 何を変えたか

変えたのはモデルだけ。重みが内蔵 RAM に入り、
推論は 41 → 3.55 ms

ESC-10 版（既定、動作例1）

モデル YAMNet-1024（ST model zoo）

知らせる音 犬・泣き声・くしゃみ
（10種のうち3種）

重み 3.1 MiB・外部フラッシュ

推論 **41.1 ms**

FSD50K 版(動作例2)

モデル YAMNet-256（ST model zoo）

知らせる音 ノック・ガラス・泣き声・話し声
（5種のうち4種）

重み 148 KB・内蔵 RAM（AXISRAM4）

推論 **3.55 ms**

変えていないもの：音の経路・前処理・タスク構成・判定規則・ログ・検証手順。
int8 化の定数はモデルから読むので、切替は AED_MODEL の 1 行

7-2 結果：基盤は同じ、モデルの弱点が出た

「その他」の無いモデルは、知らない音を全部ガラスかノックにする

	ESC-10 版	FSD50K 版
自己検証・前処理の一致	通過・差 0	通過・差 0
音の取りこぼし	0	0
対象の音のクリップ	4/6	5/6 (ノック 2/2、話し声 2/2)
対象外の生活音の誤報	0/9	9/10 (ガラス・ノックと判定)
放置中の通知	3.5 件/時	250 件/時
暗騒音の窓で確率 0.7 以上	2.6%	66%

暗騒音は音量の条件で止まる。
しかし食器・咳・椅子など実際に鳴った音は暗騒音より大きいので判定まで進み、そこでモデルがガラスかノックと誤答

既定は ESC-10 版。FSD50K版は同梱して切替可能にした。

→「その他」クラス付きの版

8 動作例：屋内音の検知と通知

基盤を実際の部屋で動かした。使えるかを決めたのはモデルだった



犬の鳴き声

約 1.6 秒



画面全体を音の色に（模式図）



赤 LED が 1 秒点灯
(検出が続くあいだは点いたまま)

UART に JSON を 1 行

```
{"win":104,"cls":"dog","p":0.99,"lat_ms":112,
"under":0,"over":0,"late":0}
```

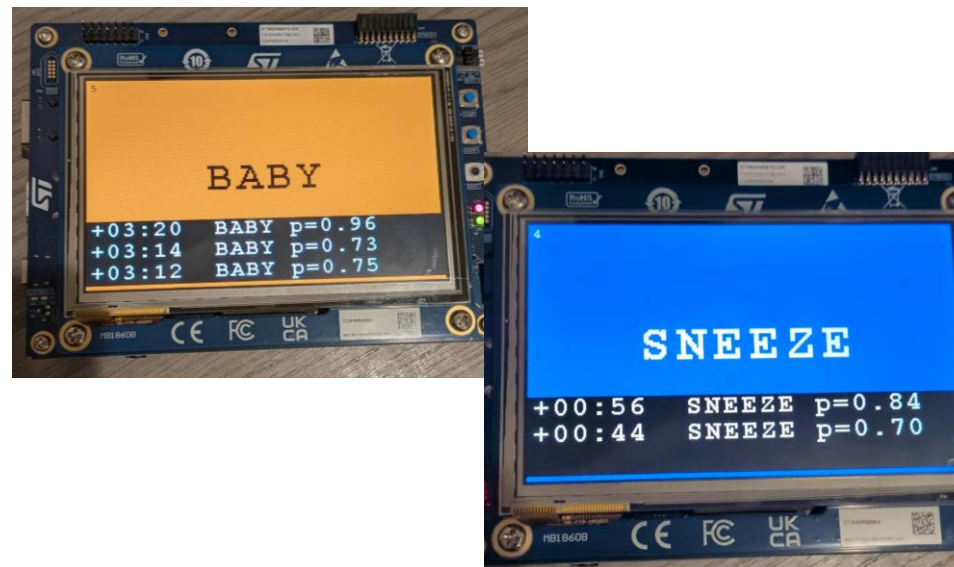


8-1 ESD-10版の動作例

音から平均 1.6 秒で、画面全体を音の種類の色に変えて知らせる



待機：READY と、1 秒ごとに変わる数字
(生存表示)



通知：画面全体を音の色に（3 秒）
履歴：直近 3 件を画面の下に残す

LED：通知で赤が 1 秒点灯(前述)、
緑の点滅はマイコン動作中の印

UART：通知ごとに JSON を 1 行（スマホ通知など外部連携の口）

8-1 動画



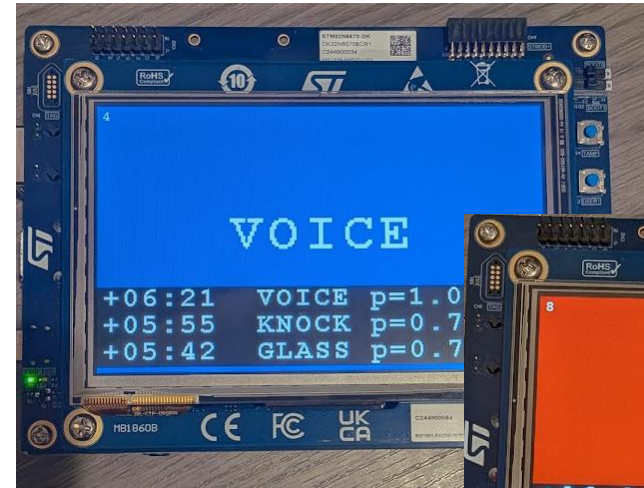
犬:Dog



咳:SNEEZE

8-2 FSD50K版の動作例

ESC-10版と基本は同様だが
ターゲットが4種類に増えた



色



犬→ノック



ガラス



泣き声
(同一だが単語変更)



咳→話し声

8-2 動画



涙:VOICE



発話:VOICE

8-3 使えるものだったか

どちらの版もそのままでは製品にならない。足りないのはモデル



- ESC-10 版は誤報は少ないが、犬、泣き声、くしゃみは部屋の知らせとして弱い
- FSD50K 版は知らせる音は合うが、誤報が多すぎる
- 2つの版を同じ基盤・同じ手順で測れたので、弱いのがモデルだと数字で分かった

判定・通知・時間の保証はそのまま使える。替えるのはモデルだけ

9 限界と展開

足りないのはモデルと時間。

基盤はそのまま他の用途に使える

限界

モデルの問題（2 つ）
知らせる音が用途に合わない／音量の判定は設置場所
しだい

時間切れの問題（4 つ）
マイクの飽和・遅延 1～2 秒・省電力・FSD50K 版は
デバグ起動のみ



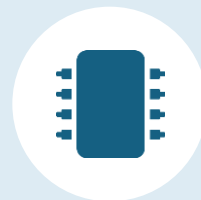
展開



機械の異音監視



見守り・防犯



既存の制御装置

落とせない周期処理に推論を足す装置なら、同
じ基盤と同じ確かめ方が使える



9-1 限界と次の一手

上 2 つはモデル、下 4 つは時間切れ。基盤の主張には影響しない

	限界	次の一手
モデル	知らせる音が用途に合わない／4 種類まで	「その他」クラス付きのモデルへ (切替の仕組みは済み)
	音量による判定は設置場所しだい	背景音を学習したモデル
時間切れ	10 cm ではマイクが飽和する	設置距離の目安を手順書に。ゲインの切替
	遅延 1～2 秒	窓をずらす間隔を縮めて約半分に
	省電力は未計測	空き 88% の間は待機命令 (WFI)
	FSD50K 版はデバッグ起動のみ	重みを実行像に入れ、ROM 領域を広げる

基盤の主張（音を落とさずに推論と表示を同居させ、壊れ方が設計どおり）は、これらの影響を受けない

9-2 今後の展開：音は「落とせない周期処理」の代表

落とせない周期処理に推論を足し、壊れないことを数えて示せる



回転機械・ポンプの異音 監視

サンプルの取りこぼし 0 を数えて示せる。暗騒音との差で切る判定は、機械ごとの定常音に自動で合う



工具の欠け・漏れ音・ 古い設備の後付け監視

配線に触らずマイクだけで足せる。1～2秒で止められる



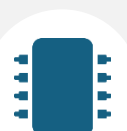
打音検査・補聴器

計測器として使った音の折り返しが、そのまま製品の機能になる



一人暮らしの見守り・窓 とドアの防犯

転倒音・呼び声、ガラスの破損・ノック。JSON で字幕装置やスマホへ



既存の制御装置に NPU 推論を足す

モータ制御やセンサの周期が上にある装置で、実験 1 の 3 条件がそのまま確かめ方になる

共通するのは

**落とせない周期処理と推論を同居させ
壊れないことを数えて示せること。
モデルは用途ごとに替えられる**

10 公開と再現

ソースと手順書・ログ・スクリプトを公開し、誰でも確かめられる



GitHub リポジトリ (タグ submit)



ソース (自作部分は Apache-2.0)



docs/manual.md 手順書



docs/logs/ 本資料の数字のもののログ



scripts/ ログ取得・集計・判定の再生



docs/third_party.md 既存ソフトとライセンス



手順書どおりに動かせば、起動ログで4段の確認が見える

→ 10-2



本資料の数字は、同梱のログからスクリプトで作り直せる

→ 4-2



判定の規則はPCで再生して、板を回さずに試せる

→ 6-2

この章の流れ

10-1 手順



10-2 起動ログ



10-3 走行ログ



10-4 最終集計

10-1 手順：取得から起動確認まで

5 つの手順で、起動確認のログまでたどり着ける



必要なもの

STM32N6570-DK と USB Type C ケーブル	STM32CubeIDE 2.2.0	Python 3 (uv)	重みの hex : STM32N6-GettingStarted-Audio v2.3.0	自己テストと通し試験に ESC-50
---------------------------------	--------------------	---------------	---	--------------------

詳しい手順と詰まりやすい点は [docs/manual.md](#)。モデルを作り直すときだけ ST Edge AI Core 4.0.1 を使う

10-2 起動ログの読み方

READY の前に、4 段の確認の結果がすべて出る

PowerShell — scripts¥log.ps1 -Port COM3 -Timestamp (時刻と途中の行を省略)

```
1 CONFIG: A production (INFER_PRI0_INVERT=0 INFER_SLOW_X=1 task_infer pri 15 model=esc10)
extflash init (MX66UW1G45G, XSPI2, DTR-OPI) ...
JEDEC ID: c2 81 3b
[ OK ] Memory-mapped mode
2 [probe] FNV-1a 0x568e47bd (expect 0x568e47bd) MATCH
npu hw init (AXISRAM3-6, NPU, CACHEAXI, RIF)
2 [ram] AXISRAM6 @ 0x34350000: MATCH (0 mismatches in 64 words x 2 patterns)
2 [npu] no fault. VERSION=0x01b4121f ... expect 0x01b4121f -> MATCH
3 notify: 3 target classes (dog crying_baby sneezing), threshold p>0.70, peak gate -30 dBFS
3 notify: rel gate rms >= floor + 10 dB, floor = p10 of the last 60 windows' rms
npu repeat x10 (same input):
4 [10] 40696 us, max diff vs first = 0 x1e-4 (bit-identical)
inference time: min=40675 mean=40696 max=40709 us (10 of 10 runs OK)
totals: same 12288 diff 0 of 12288, max |d| 0, |d|>1: 0
5 preproc test PASS (no |d| > 1, and top1 from the board tensor matches PC)
6 READY 起動確認おわり。ここから本番
```

1 CONFIG

実験の条件 (A・B・D) と、使うモデル

2 MATCH

フラッシュ・RAM・NPU を実際に読んで一致

3 notify

判定の規則 (確率・音量・暗騒音との差)

4 bit-identical

同じ入力で 10 回推論し、結果が完全に同じ

5 preproc test PASS

前処理が PC と 12,288 個すべて一致 (参照が無ければ SKIP)

6 READY

確認が終わった印。合否は 2～5 の行で見る

10-3 走行ログの読み方

1 窓ごとに、音量・判定・遅れが行として残る

走行中の出力（通し試験で犬のクリップを流したところ。時刻と一部の行を省略）

```
1 in=16128Hz out=16000Hz ring=1200 under=0 over=0 (dt=1000000us) loglag=119563us
2 {"win":104,"cls":"dog","p":0.99,"lat_ms":112,"under":0,"over":0,"late":0}
3 win 104 pos= 1597440 -11dBFS [#####.....] rms= 9429 peak=32768 seam=ok lag=334us
4 -> dog p=0.99 (floor -44.8 Δ+34.0) preproc=69225us infer=41093us
   lcd: win 104 -> DOG p=0.99 (23611 us after notify)
```

1 in=…Hz

周期ハンドラが 1 秒ごとに出す統計。入力 of の速さを取りこぼし

3 win 行

窓の番号・平均音量（dBFS）・瞬間音量（peak）・窓が揃ってから処理を始めるまでの遅れ（lag）

2 JSON

外部連携用の通知。lat_ms は検出から通知までの時間

4 -> 行

判定：クラス・確率・暗騒音（floor）との差（Δ）と、前処理・推論の時間

10-4 最終集計（FINAL）の読み方

取りこぼしと「捨てた数」がすべて 0 なら、何も落ちていない

1 時間運転の最後に出る FINAL (2026-09-27 17:20)

```
1 tap final: windows=3854 (expected 3854 from 59209472 samples) overrun=0 torn=0 skipped=0
2 audio under=0 over=0 late=0 | aed run=3854 err=0 (last er=0)
  preproc max=69145us avg=69077us | infer max=41158us avg=41054us
3 notify out=157 held=3436 offlist=261 gated=427 lat max=112328us (0 lower bounds)
4 log sent=24238 dropped=0 lag max=127874us
4 disp sent=83 dropped=0 max=98707us avg=30502us
```

1 windows = expected

音の量から計算した窓の数と、推論した窓の数が一致
→ 窓の消失なし

3 notify out / gated

JSON を出した数と、音量の条件で止めた窓の数

2 audio under / over / late

音の取りこぼし。3 つとも 0 が合格

4 log / disp dropped

捨てたログの行と画面の更新。0 が合格

11 まとめ

音・推論・表示を優先度で並べ、
壊しても音だけは生き残ることを実測で示した

1時間、止めずに動かしても

0 回

音の取りこぼし

3,854 窓すべて。ログ 24,238 行・画面 83
回も欠け 0

推論を音より優先にすると

8%

入力の欠落

推論と表示は無事。順序を間違えると
音が壊れる

推論を 10 倍重くても

0 回

音の取りこぼし

窓 13% 消失・ログ 93% 破棄・画面 10 分
停止でも音は無事

⇄ モデルは交換部品。2 つのモデルを同じ手順で測り、基盤の数字は変わらなかった（7 章）

📉 落とせない周期処理に推論を足す装置の土台と、その確かめ方として使える（9 章）

付録 A1 既存ソフトウェア一覧

既存のソフトはすべて出典とライセンスを明記した

名称	権利者	ライセンス	改変
μ T-Kernel 3.0 BSP2	TRON Forum	T-License 2.2 (一部 2.1)	無改変（リンカスクリプトに追記）
STM32N6 HAL / CMSIS	STMicroelectronics / Arm	BSD-3 / Apache-2.0	無改変 (HAL の LTDC ドライバ 4 本を追加)
ll_aton + NetworkRuntime 1201	STMicroelectronics	SLA0104	無改変 (4.0.1 同梱版)
生成済みネットワーク (ESC-10・FSD50K)	STMicroelectronics	SLA0104	無改変。ESC-10 の重み hex (SLA0044) は同梱しない
MX66UW1G45G ドライバ / ExtMem Manager	STMicroelectronics	BSD-3 / SLA0044	無改変（設定ファイルに本機の値）
RK050HR18 定義・Font24 / FSD50K 版の tflite	STMicroelectronics	BSD-3 / Apache-2.0	無改変
ESC-50（試験用クリップ）	K. Piczak	CC BY-NC 3.0	同梱しない（スクリプトで取得・生成）

付録 A2 既存・改変・自作の区別

既存のものはそのまま使い、無かった部分を自作した

層	既存のまま	改変	自作
OS・HAL	μ T-Kernel 3.0 BSP2、STM32N6 HAL / CMSIS	CubeMX 生成の main.c / it.c / hal_conf.h（クロック・割込み）とリンクスクリプト	フォールト表・トレース・ログ基盤
音	WM8904 コーデックの設定（ST 由来）	—	PCM FIFO、窓リング、ヘッドホンへの折り返し、窓の切り出し
推論	ll_aton + NetworkRuntime、生成済みネットワーク（ST Edge AI Core）	—	log-mel 前処理、NPU の初期化、自己検証
出力	HAL LTDC、RK050HR18 定義、Font24	—	LTDC 直結の LCD ドライバ、通知の判定、LCD タスク、JSON

付録 A3 ST リファレンス実装との違い

OS の上で音と推論を分け、壊れ方まで測った

観点	ST GettingStarted (ベアメタル)	本作品
OS	無し (メインループ)	μ T-Kernel 3.0、7 タスク + 周期 / アラームハンドラ
音の経路	取り込みと推論が同じループ	取り込みは最優先のタスク。 推論が遅れても落ちない (実験 1)
前処理	ST のライブラリ	自作の log-mel (PC と 1 ビットも違わない。PC 側スクリプト同梱)
検証	デモ動作	自己検証・前処理の一致・対照試験・1 時間運転・ 誤報の再走行、ログと再生スクリプト
表示・通知	無し	LTDC 直結の LCD (全面の色 + 履歴)、LED、UART JSON
低レベルの初期化	GettingStarted に含まれる	BSP2 に無かった分を自作 (フラッシュ・NPU・RIF・LCD)